Computertechnik

In der Prüfung "Computertechnik und Programmieren" ist nur die unten angehängte Formelsammlung zulässig!

1. Zahlen und Zeichen

1.1. Daten als Bits und Bytes

1.1.1. Binärdarstellung

1 - High Pegel - logisch wahr
0 - Low Pegel - logisch falsch

1.1.2. Hexadezimaldarstellung

Zusammenfassen von 4 Bits zu 0, 1, ..., E,D F

0: 0000	1: 0001	8: 1000	9: 1001
2: 0010	3: 0021	A: 1010	B: 1011
4: 0100	5: 0101	C: 1100	D: 1101
6: 0110	7: 0111	E: 1110	F: 1111

1.1.3. Bits/Bytes (MMIX)

- 1 Byte = 8 Bits = 1 Byte
- 2 Byte = 16 Bits = 1 Wyde
- 4 Byte = 32 Bits = 1 Tetra
- 8 Byte = 64 Bits = 1 Octa

1.1.4. ASCII-Tabelle

	0x0...	0x1...	0x2...	0x3...	0x4...	0x5...	0x6...	0x7...
...0	NUL	DLE	SP	0	@	P	`	p
...1	SOH	DC1	!	1	A	Q	a	q
...2	STX	DC2	"	2	B	R	b	r
...3	ETX	DC3	#	3	C	S	c	s
...4	EOT	DC4	$	4	D	T	d	t
...5	ENQ	NAK	%	5	E	U	e	u
...6	ACK	SYN	&	6	F	V	f	v
...7	BEL	ETB	'	7	G	W	g	w
...8	BS	CAN	(	8	H	X	h	x
...9	HT	EM	)	9	I	Y	i	y
...A	NL	SUB	*	:	J	Z	j	z
...B	VT	ESC	+	;	K	[	k	{
...C	NP	FS	,	<	L	\	l	|
...D	CR	GS	-	=	M	]	m	}
...E	SO	RS	.	>	N	^	n	~
...F	SI	US	/	?	O	_	o	DEL

© Michael Zwick

1.2. Zahlen

1.2.1. Vorzeichenlose Festkommazahlen

Definition: $v = (a_{n-1} \cdot b^{n-1} + \cdots + a_1 \cdot b^1 + a_0 \cdot b^0) \cdot b^r$
b: Basis (meist 2); n: Stellenzahl; $a \in 0 \ldots (b-1)$; r: Position des Kommas

1.2.2. Einer Komplement

Vorzeichen wird durch MSB abgelesen
Umwandlung durch invertieren aller Bits

1.2.3. Zweier Komplement

Vorzeichen wird durch MSB abgelesen
Umwandlung durch invertieren aller Bits und binärer Addition des Werts 1 (von positiv zu negativ)

1.2.4. Gleitkommazahlen nach IEEE 754

Bestehend aus: Vorzeichen (s), Exponentteil (e), Mantisse (f)
Vorgehen (Bsp. $17,625$):

1. Zahl vor dem Komma $(17,0)$ und nach dem Komma $(0,625)$ getrennt in binär umwandeln
2. Zusammen in binär schreiben $(10001,101)$
3. Das Komma durch Anpassung des Exponenten $e-K$, verschieben, um das Format $1,f$ zu erhalten $(1,0001101 \cdot 2^4 \rightarrow e-K=4)$

Definition (normalisiert): $v = (-1)^s \cdot 1,f \cdot 2^{e-K}$

32 Bit:

- $K = 127$
- $s = 1$ Bit
- $e = 8$ Bit
- $f = 23$ Bit

64 Bit:

- $K = 1023$
- $s = 1$ Bit
- $e = 11$ Bit
- $f = 52$ Bit

Spezialfälle (de-normalisiert $v = (-1)^s \cdot 0,f \cdot 2^{e-K}$):

- 0: $e = 0$; $f = 0$
- $+\infty$: $e = 1 \ldots 1$; $f = 0$; $s = 0$
- $-\infty$: $e = 1 \ldots 1$; $f = 0$; $s = 1$
- Not a Number: $e = 1 \ldots 1$; $f > 0$

2. Arithmetische Schaltungen

2.1. Schaltungselemente

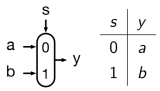
x	y	Inverter $\overline{x}$	AND $x \wedge y$	OR $x \vee y$	XOR $x \oplus y$	NAND $\overline{x \wedge y}$	NOR $\overline{x \vee y}$
0	0	1	0	0	0	1	1
0	1	1	0	1	1	1	0
1	0	0	0	1	1	1	0
1	1	0	1	1	0	0	0

Ein *Treiber* besteht aus zwei Invertern in Serie geschalten.

2.2. Fan-In/Fan-Out

Fan-In: max. Eingänge in Gatter
Fan-Out: max. Ausgänge aus Gatter

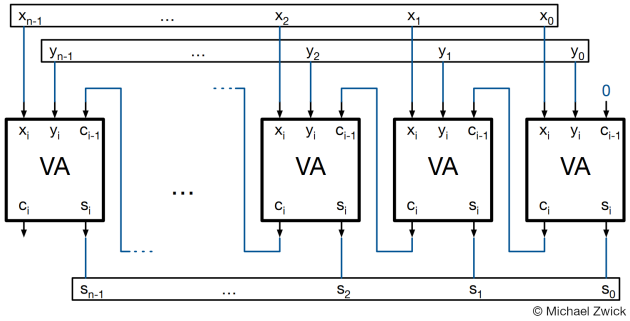
2.3. Multiplexer



s	y
0	a
1	b

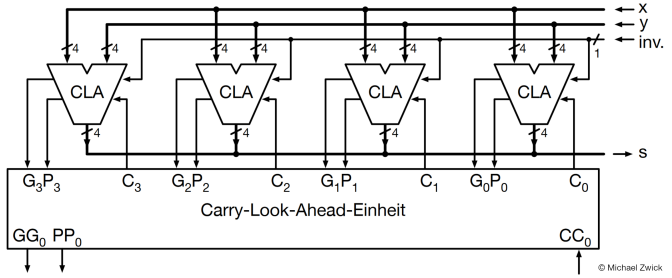
2.4. Ripple-Carry-Addierer

Berechnung der Summe aus zwei Summanden



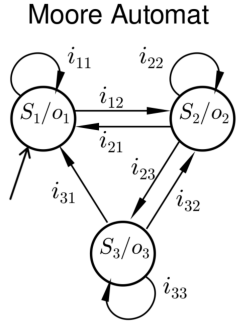
2.5. Carry-Look-Ahead

Schnellere Berechnung der Summe aus zwei Summanden durch zusätzliche Logik zur Bestimmung des Übertrags



Überlauf (c_i) eines Volladdierers:
$c_i = g_i \vee (p_i \wedge g_{i-1}) \vee (p_i \wedge p_{i-1} \wedge g_{i-2}) \vee \ldots$

2.6. Zustandsautomat



Input: Eingangssignale in den Automaten

Output: Taktleitungen/ Clock-Signale (0 oder 1) und andere Signale (0, 1, X)

2.7. ROM-Steuerung

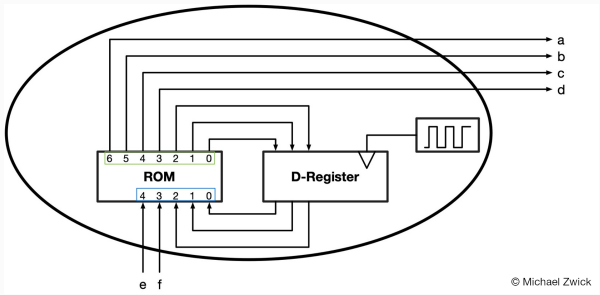
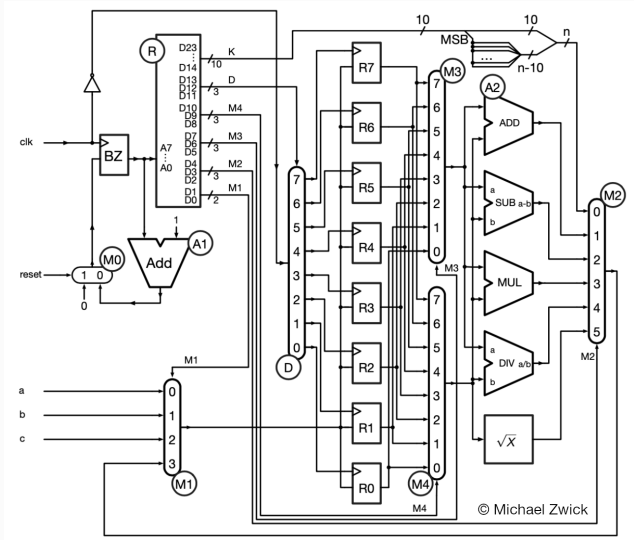


Tabelle mit Eingabe (gegeben) und Ausgabe (auszufüllen)
Vorgehen: Einträge der gegebenen Tabelle in Binärzahlen umwandeln. Bits $0, 1, 2$ der Eingabe sind der derzeitige Zustand. Bits $3, 4, \ldots$ der Eingabe sind die Eingangssignale für den nächsten Zustand. Bits $0, 1, 2$ der Ausgabe sind der Folgezustand und Bits $3, 4, \ldots$ die Ausgangssignale des aktuellen Zustands.
Ansatz:

e f	Zst.	a b c d	Folgezst.
(4 3)	(2 1 0)	(6 5 4 3)	(2 1 0)
...	...	...	...
...	...	...	...

2.8. Universalrechner



Auch hier gilt: Klammer *vor* Wurzel *vor* Punkt *vor* Strich!

- **K:** Konstante
- **D:** Ziel-Register
- **M1:** Ergebnis-Auswahl
- **M2:** Rechenoperation/Konstante-Laden
- **M3:** Operand 1 *(bei $\sqrt{x}$ jedoch M4 verwenden)*
- **M4:** Operand 2

3. MMIX-Architektur

3.1. Big-Endian und Little-Endian

Big-Endian: höherwertigeres Byte wird adressiert (von links nach rechts)
Little-Endian: niederwertigeres Byte wird adressiert (von rechts nach links)

3.2. Übersetzungstabelle

	0x..0	0x..1	0x..2	0x..3	0x..4	0x..5	0x..6	0x..7	
0x0..	TRAP 5v	FCMP v	FUN v	FEQL v	FADD 4v	FIX 4v	FSUB 4v	FIXU 4v	0x0..
	FLOT[I] 4v		FLOTU[I] 4v		SFLOT[I] 4v		SFLOTU[I] 4v		
0x1..	FMUL 4v	FCMPE 4v	FUNE v	FEQLE 4v	FDIV 40v	FSQRT 40v	FREM 4v	FINT 4v	0x1..
	MUL[I] 10v		MULU[I] 10v		DIV[I] 60v		DIVU[I] 60v		
0x2..	ADD[I] v		ADDU[I] v		SUB[I] v		SUBU[I] v		0x2..
	2ADDU[I] v		4ADDU[I] v		8ADDU[I] v		16ADDU[I] v		
0x3..	CMP[I] v		CMPU[I] v		NEG[I] v		NEGU[I] v		0x3..
	SL[I] v		SLU[I] v		SR[I] v		SRU[I]v		
0x4..	BN[B] v+π		BZ[B] v+π		BP[B] v+π		BOD[B] v+π		0x4..
	BNN[B] v+π		BNZ[B] v+π		BNP[B] v+π		BEV[B] v+π		
0x5..	PBN[B] 3v-π		PBZ[B] 3v-π		PBP[B] 3v-π		PBOD[B] 3v-π		0x5..
	PBNN[B] 3v-π		PBNZ[B] 3v-π		PBNP[B] 3v-π		PBEV[B] 3v-π		
0x6..	CSN[I] v		CSZ[I] v		CSP[I] v		CSOD[I] v		0x6..
	CSNN[I] v		CSNZ[I] v		CSNP[I] v		CSEV[I] v		
0x7..	ZSN[I] v		ZSZ[I] v		ZSP[I] v		ZSOD[I] v		0x7..
	ZSNN[I] v		ZSNZ[I] v		ZSNP[I] v		ZSEV[I] v		
0x8..	LDB[I] μ+v		LDBU[I] μ+v		LDW[I] μ+v		LDWU[I] μ+v		0x8..
	LDT[I] μ+v		LDTU[I] μ+v		LDO[I] μ+v		LDOU[I] μ+v		
0x9..	LDSF[I] μ+v		LDHT[I] μ+v		CSWAP[I] 2μ+2v		LDUNC[I] μ+v		0x9..
	LDVTS[I] v		PRELD[I] v		PREGO[I] v		GO[I] 3v		
0xA..	STB[I] μ+v		STBU[I] μ+v		STW[I] μ+v		STWU[I] μ+v		0xA..
	STT[I] μ+v		STTU[I] μ+v		STO[I] μ+v		STOU[I] μ+v		
0xB..	STSF[I] μ+v		STHT[I] μ+v		STCO[I] μ+v		STUNC[I] μ+v		0xB..
	SYNCD[I] v		PREST[I] v		SYNCID[I] v		PUSHGO[I] 3v		
0xC..	OR[I] v		ORN[I] v		NOR[I] v		XOR[I] v		0xC..
	AND[I] v		ANDN[I] v		NAND[I] v		NXOR[I] v		
0xD..	BDIF[I] v		WDIF[I] v		TDIF[I] v		ODIF[I] v		0xD..
	MUX[I] v		SADD[I] v		MOR[I] v		MXOR[I] v		
0xE..	SETH v	SETMH v	SETML v	SETL v	INCH v	INCMH v	INCML v	INCL v	0xE..
	ORH v	ORMH v	ORML v	ORL v	ANDNH v	ANDNMH v	ANDNML v	ANDNL v	
0xF..	JMP[B] v		PUSHJ[B] v		GETA[B] v		PUT[I] v		0xF..
	POP 3v	RESUME 5v	[UN]SAVE 20μ+v		SYNC v	SWYM v	GET v	TRIP 5v	
	0x..8	0x..9	0x..A	0x..B	0x..C	0x..D	0x..E	0x..F	

© Michael Zwick

Um Opcode zu erhalten den Befehl in der Tabelle suchen. Erste Hex-Zahl links ablesen, zweite Hex-Zahl (je nach Subzeile) oben oder unten ablesen. I: immideate (Direktoperand); B: backwards (bei Sprungbefehlen)

3.3. Spezielle Befehle

3.3.1. Namensraum

Namensraum eröffnen: `PREFIX Name`
Namensraum beenden: `PREFIX :`

3.3.2. Stack-Pointer (SP)

- n Register auf Stack ablegen:
  1. SP anpassen: `SUB, :SP, :SP, 8*n`
  2. Mit `STO` speichern
- n OCTAs aus Stack einlesen:
  1. Mit `LDO` einlesen
  2. SP anpassen: `ADD, :SP, :SP, 8*n`

3.4. Alignment

3.4.1. In den Speicher

- **BYTE:** geht immer
- **WYDE:** letztes Bit muss 0 sein
- **TETRA:** letzten beiden Bits müssen 0 sein
- **OCTA:** letzten drei Bits müssen 0 sein

Ist dies nicht der Fall werden die betroffenen Bits aufgrund des Alignments beim Speicherprozess automatisch auf 0 gesetzt

3.4.2. Laden in Register

Beim Laden wird das Register so aufgefüllt, dass das LSB der geladenen Bitfolge auch das LSB des Registers ist.
ACHTUNG: Das MSB der vordersten Hex-Zahl entscheidet bei signed Operationen ob der Rest links davon mit 0 ... 0 oder F ... F aufgefüllt wird.

3.5. Pipelining

3.5.1. Konflikte

- **Datenkonflikt:** Zugriff auf ein Register das noch gespeichert werden muss
- **Strukturkonflikt:** Zugriff von zwei Befehlen auf die selbe Hardware-Resource
- **Steuerungskonflikt:** Bei Sprungsbefehlen werden ggf. darauffolgende Befehle fälschlicherweise geladen, wenn noch nicht klar ist ob gesprungen werden soll oder nicht

3.5.2. Datenpfad

Vorgehen: von rechts nach links die jeweiligen Inhalte der Pipelining-Register eintragen. Dabei ggf. Änderungen von Werten berücksichtigen.

3.6. Cache

3.6.1. Allgemeines

Hit-Rate + Miss-Rate = 100%
Mittlere Zugriffszeit: (Hit-Time) · (Hit-Rate) + (Miss-Time) · (Miss-Rate)
Arbeitsspeicher-Adresse: Schlüssel + Rahmen-Nr + Byte-Auswahl
Cache-Inhalt: (Rahmen-Nr + Byte-Auswahl)
Tag-RAM: (Schlüssel + Rahmen-Nr)
Merke: 2^{10} Byte = 1kB und 2^{20} Byte = 1MB

3.6.2. Direkt-Abgebildet

Es existiert pro Rahmen-Nr. nur **ein Rahmen** in dem eine Byte-Folge gespeichert werden kann
Problem: Man kann nicht zwei verschiedene Byte-Folgen mit unterschiedl. Schüsseln und selber Rahmen-Nr. speichern

3.6.3. Voll-Assoziativ

Eine Bytefolge kann in jedem Rahmen gespeichert werden
Problem: Schlüsselvergleich dauert lange

3.6.4. Set-Assoziativ

Es existieren pro Rahmen-Nr. mehrere Rahmen, in denen eine Byte-Folge gespeichert werden kann (Set)
Vorteil: man kann zwei verschiedene Byte-Folgen mit selber Rahmen-Nr. speichern und ist trotzdem noch relativ schnell

4. Mikrocontroller-Programmierung

4.1. Das Board

Anschluss: 2x USB für Programmierung und Spannungsversorgung
AVR-Programmieradapter
Steckplatine mit V_{dd} und GND Anschlüssen

4.2. Übersetzungsbefehle

- Übersetzen: `make`
- Übersetzen und in den Controller laden: `make program`
- Übersetzen und Assembler-Code zeigen: `make show`
- Übersetzte Dateien löschen: `make clean`
- Taktfrequenz u.a. einstellen: `make fuses`

4.3. Bitweise Operationen (Bsp: PORTD)

- Bit Nr. 0 setzen: `PORTD |= 1` (bitweise ODER)
- Bit Nr. 0 löschen: `PORTD &= ~1` (bitweise UND)
- Bit Nr. 0 ändern: `PORTD ^= 1` (bitweise XOR)

4.4. Wortbreitenunterschiede

Typ	herkömmlicher PC	8-Bit Mikrocontroller
char	1 Byte	1 Byte
short	2 Bytes	2 Bytes
int	4 Bytes	2 Bytes
float	4 Bytes	4 Bytes
double	8 Bytes	4 Bytes
long	8 Bytes	4 Bytes
long long	8+ Bytes	8 Bytes

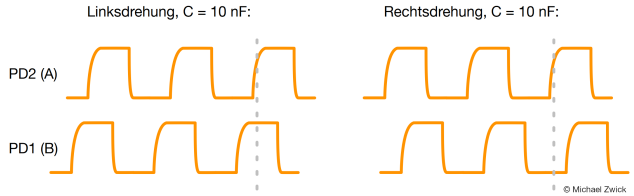
4.5. Interrupts

Beim Auftreten eines Interrupts wird aus dem derzeit ausgeführten Code in die Interrupt-Service-Routine (ISR) gesprungen. Dieses Verhalten kann genutzt werden, um z.B. beim Drücken eines Tasters oder nach einer bestimmten Zeit eine Funktion aufzurufen. Dabei muss nicht ständig überprüft werden, ob ein Interrupt auftritt (das springen in die ISR geschieht automatisch).

4.5.1. Interrupt mit Taster

1. Gewünschten Pin als Eingang konfigurieren
2. Interrupts global freischalten
3. Spez. Interrupt im EIMSK-Register freischalten
4. Im EICRA konfigurieren, welcher Pegelzustand des spez. Interrupt berücksichtigt wird (z.B. steigende Flanke)
5. ISR-Funktion schreiben:
   `ISR(INT`n`_vect) {`
   `...`
   `}`

4.5.2. Drehgeber



2 Pins werden benötigt (ein Interrupt und ein normaler Eingang)
Pin A: Feststellen, dass gedreht wurde (Interrupt).
Pin B: Feststellen, in welche Richtung gedreht wurde (Eingang auslesen).
Zuerst wird in die ISR vom Interrupt (A) gesprungen. Dort wird dann folgendes geprüft:

- hat B High-Pegel? - Dann war es eine Linksdrehung
- hat B Low-Pegel? - Dann war es eine Rechtsdrehung

4.5.3. Timer-Interrupts

Der Zähler `TCNT`n wird mit der Frequenz $f = \frac{\text{clk}}{\text{Prescaler}}$ um 1 erhöht.
Überlauf-Interrupt, wenn `TCNT`n...

- von $2^8 - 1$ auf $2^8 = 256$ erhöht wird (bei $n \in \{0, 2\}$)
- von $2^{16} - 1$ auf $2^{16} = 65536$ erhöht wird (bei $n \in \{1, 3\}$)